

Desarrollar una aplicación
para smartwatch

Templates

Phone and Tablet

Wear OS

Television

Automotive



No Activity



Empty Wear App With Tile And Comp...



Empty Wear App

Previous

Next

Cancel

Finish

manifests

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <manifest xmlns:android="http://schemas.android.com/apk/res/android">
3
4     <uses-permission android:name="android.permission.WAKE_LOCK" />
5     <uses-permission android:name="android.permission.BLUETOOTH" />
6     <uses-permission android:name="android.permission.BLUETOOTH_ADMIN" />
7     <uses-permission android:name="android.permission.BODY_SENSORS"/>
8
9     <uses-feature android:name="android.hardware.type.watch" />
10
11     <application
12         android:allowBackup="true"
13         android:icon="@mipmap/ic_launcher"
14         android:label="@string/app_name"
15         android:supportsRtl="true"
16         android:theme="@android:style/Theme.DeviceDefault">
17         <uses-library
18             android:name="com.google.android.wearable"
19             android:required="true" />
```

- En res crear un nuevo directorio llamado layout
- Dentro de layout crear un layout resource file llamado activity_main
- Cambiar

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
3     android:orientation="vertical"
4     android:layout_width="match_parent"
5     android:layout_height="match_parent">
6
7 </LinearLayout>
```

- por:
- además agregar la
línea 5
- se tendrá que
agregar la dependencia
y sincronizar el proyecto

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <androidx.constraintlayout.widget.ConstraintLayout
3     xmlns:android="http://schemas.android.com/apk/res/android"
4     android:orientation="vertical"
5     xmlns:app="http://schemas.android.com/apk/res-auto"
6     android:layout_width="match_parent"
7     android:layout_height="match_parent">
8
9 </androidx.constraintlayout.widget.ConstraintLayout>
```

agregar un botón

```
1  <Button
2      android:id="@+id/boton"
3      android:layout_width="wrap_content"
4      android:layout_height="wrap_content"
5      android:text="Saludar"
6      app:layout_constraintBottom_toBottomOf="parent"
7      app:layout_constraintEnd_toEndOf="parent"
8      app:layout_constraintStart_toStartOf="parent"
9      app:layout_constraintTop_toTopOf="parent"
10 />
```

En el MainActivity.kt

```
1  class MainActivity : ComponentActivity() {  
2      override fun onCreate(savedInstanceState: Bundle?) {  
3          installSplashScreen()  
4  
5          super.onCreate(savedInstanceState)  
6          setContentView(R.layout.activity_main);  
7  
8          val boton:Button=findViewById(R.id.boton);  
9          val context=this;  
10         boton.setOnClickListener {  
11             Toast.makeText(context, "Alerta", Toast.LENGTH_SHORT).show();  
12         }  
13     }  
14 }  
15 }
```

Segunda ventana

- Crear un segundo archivo layout resource file llamado prueba con la misma configuración que el anterior con otro botón y un text view

```
1  <TextView
2      android:id="@+id/texto"
3      android:layout_width="wrap_content"
4      android:layout_height="wrap_content"
5      android:text="Soy la ventana 2"
6      app:layout_constraintBottom_toBottomOf="parent"
7      app:layout_constraintEnd_toEndOf="parent"
8      app:layout_constraintStart_toStartOf="parent"
9      app:layout_constraintTop_toTopOf="parent"
10 />
```


- En presentation crean un Kotlin class llamado Prueba que herede de ComponentActivity

```
1  package com.example.myapplication.presentation
2
3  import android.os.Bundle
4  import androidx.activity.ComponentActivity
5  import com.example.myapplication.R
6
7  class Prueba: ComponentActivity(){
8
9      override fun onCreate(savedInstanceState: Bundle?) {
10          super.onCreate(savedInstanceState)
11
12          setContentView(R.layout.prueba)
13      }
14  }
```

manifests

agregamos otro activity al manifest

```
1  <activity android:name=".presentation.Clase2"></activity>
```

- En el archivo [MainActivity.kt](#) en la función onCreate agregamos el siguiente código

```
1  val boton: Button =findViewById(R.id.boton);
2
3  boton.setOnClickListener{
4      val intent= Intent(this@MainActivity, Clase2::class.java)
5      startActivity(intent)
6  }
```

Agregar icono

agregar imagen a \app\src\main\res\mipmap-hdpi

En el manifest cambiar la propiedad icon por el nombre de la imagen

```
1  <application
2      android:allowBackup="true"
3      android:icon="@mipmap/mishito"
4      android:label="@string/app_name"
5      android:supportsRtl="true"
6      android:theme="@android:style/Theme.DeviceDefault"
7  >
```

Configurar smartwatch para emparejamiento

ajustes > acerca del reloj > informacion de software > version software (clicks)

ajustes > Opciones de desarrollador:

Depuración ADB

Depuración inalámbrica (Apagar el bluetooth del reloj para que se conecte a una red) > Vincular dispositivo nuevo

Correr aplicacion en smartwatch físico (windows)

Tools > SDK Manager > SDK Tools > Platform-Tools

Agregar a las variables de entorno “AppData\Local\Android\Sdk\platform-tools”

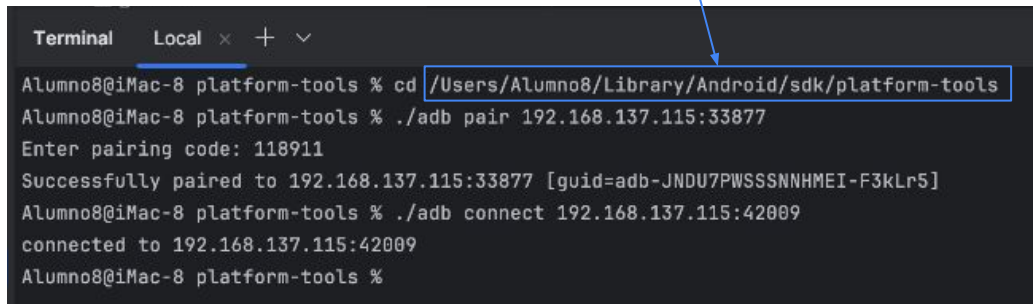
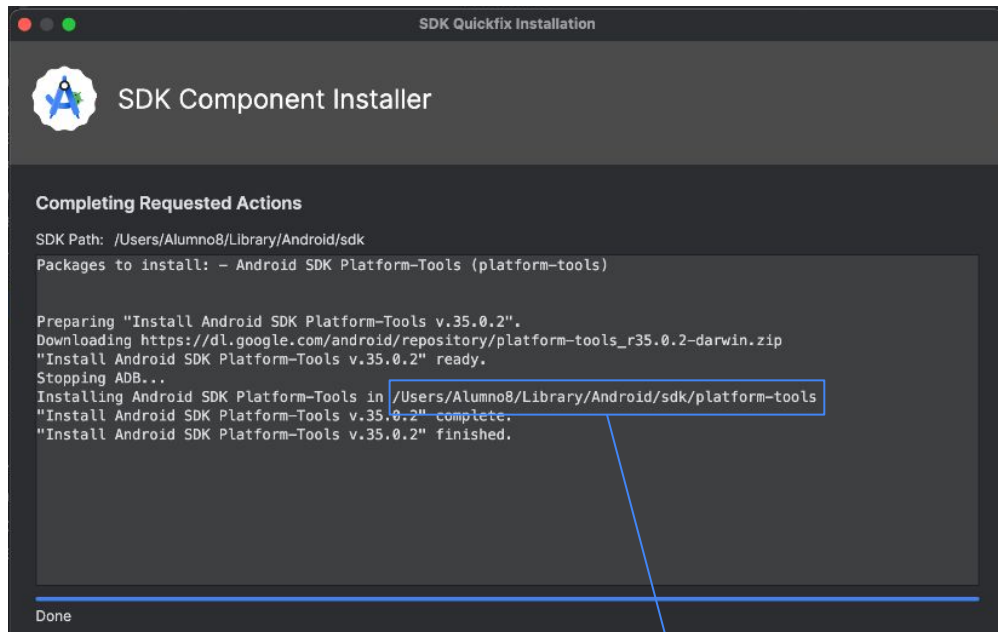
En la terminal en android correr: (sustituir ip, puerto y código)

```
PS C:\Users\celju\AndroidStudioProjects\MishiWatch> adb pair 192.168.1.14:44951
Enter pairing code: 389656
Successfully paired to 192.168.1.14:44951 [guid=adb-RFAW50GQ9ER-QB5gUm]
PS C:\Users\celju\AndroidStudioProjects\MishiWatch> adb connect 192.168.1.14:44589
connected to 192.168.1.14:44589
```

Correr aplicacion en smartwatch físico (mac)

Tools > SDK Manager > SDK Tools >
Platform-Tools

en terminal de android studio entrar a la
carpeta donde se guarda platform-tools



Instalar APK en el smartwatch

En Bugjeager primero emparejar y luego conectar

Instalar el APK

https://www.youtube.com/watch?v=Oab9GabAuxc&ab_channel=VMGChannel

Otros elementos de hardware

Vibración

Micrófono

Bocina

Vibración

Manifest

```
1 <uses-permission android:name="android.permission.VIBRATE"/>
```

Kotlin class



```
1 import android.os.Vibrator
2
3 class MainActivity : ComponentActivity(){
4
5     private lateinit var vibrator: Vibrator
6
7     override fun onCreate(savedInstanceState: Bundle?) {
8         installSplashScreen()
9         super.onCreate(savedInstanceState)
10        setContentView(R.layout.activity_main)
11
12        vibrator = getSystemService(VIBRATOR_SERVICE) as Vibrator
13
14        val boton: Button = findViewById(R.id.boton);
15
16        boton.setOnClickListener{
17            if(vibrator.hasVibrator){
18                vibrator.vibrate(500);//Vibrar por 500 milisegundos
19            }
20        }
21    }
22 }
```

Reproducir audio

Guardar el archivo mp3 en res/raw (si no existe el directorio crearlo, el archivo solo debe contener letras y minúsculas)

```
1  import android.media.MediaPlayer
2
3  class MainActivity : AppCompatActivity(){
4
5      private lateinit var mediaPlayer: MediaPlayer
6
7      override fun onCreate(savedInstanceState: Bundle?) {
8          installSplashScreen()
9          super.onCreate(savedInstanceState)
10         setContentView(R.layout.activity_main)
11
12         mediaPlayer = MediaPlayer.create(this, R.raw.archivo)
13
14         val boton: Button = findViewById(R.id.boton);
15
16         boton.setOnClickListener{
17             if(!mediaPlayer.isPlaying){
18                 mediaPlayer.start()
19             }
20         }
21     }
22 }
```

Sensores

Código de los sensores

- **Accelerometer:** mide la vibración o la aceleración del movimiento
 - `TYPE_ACCELEROMETER` `TYPE_LINEAR_ACCELERATION`
- **Barometer:** mide la presión del aire
 - `TYPE_PRESSURE`
- **Bioelectrical Impedance Analysis Sensor:** evaluar el agua corporal total y la composición corporal
 - sensor avanzado que no está cubierto por la API estándar de Android
- **Electrical Heart Sensor:** mide la frecuencia cardiaca por medio de señales eléctricas
 - sensor avanzado que no está cubierto por la API estándar de Android
- **Gyro Sensor:** mide el movimiento de rotación
 - `TYPE_GYROSCOPE`
- **Geomagnetic Sensor:** mide y cuantifica las fuerzas magnéticas (brújula)
 - `TYPE_MAGNETIC_FIELD` `TYPE_MAGNETIC_FIELD_UNCALIBRATED`
- **Light Sensor:** mide el grado de iluminación del entorno
 - `TYPE_LIGHT`
- **Optical Heart Rate Sensor:** mide la frecuencia cardiaca por medio de LEDs verdes
 - `TYPE_HEART_RATE`
- **Infrared Temperature Sensor:** mide la temperatura
 - sensor avanzado que no está cubierto por la API estándar de Android

Nota: Los sensores que no están soportados nativamente por android, se deben consultar en la [documentación](#) del fabricante

Para usar cualquier sensor

Manifests

```
1 <uses-permission android:name="android.permission.BODY_SENSORS"/>
```

```
1 package com.example.myapplication.presentation
2
3 import android.os.Bundle
4 import androidx.activity.ComponentActivity
5 import com.example.myapplication.R
6 import android.content.Context
7 import android.hardware.Sensor
8 import android.hardware.SensorEvent
9 import android.hardware.SensorEventListener
10 import android.hardware.SensorManager
11
12 class Clase: ComponentActivity(), SensorEventListener {
13     private lateinit var sensorManager: SensorManager
14     private var sensor: Sensor? = null
15     private var sensorType = Sensor.CODIGO_DEL_SENSOR
16
17     override fun onCreate(savedInstanceState: Bundle?) {
18         super.onCreate(savedInstanceState)
19
20         sensorManager = getSystemService(Context.SENSOR_SERVICE) as SensorManager
21         sensor = sensorManager.getDefaultSensor(sensorType)
22
23         setContentView(R.layout.ventana2)
24         startSensor()
25     }
26 }
```

Agregar

Cambiar dependiendo del sensor a usar

Función para pedir permisos para usar el sensor

```
1 private fun startSensor(){
2     if(ActivityCompat.checkSelfPermission(this, Manifest.permission.BODY_SENSORS)!=PackageManager.PERMISSION_GRANTED){
3         ActivityCompat.requestPermissions(this, arrayOf(Manifest.permission.BODY_SENSORS), 1001)
4         return
5     }
6     if(sensor!=null){
7         sensorManager.registerListener(this, sensor, SensorManager.SENSOR_DELAY_NORMAL)
8     }
9 }
```

```
1  override fun onSensorChanged(event: SensorEvent?) {
2      if(event?.sensor?.type==sensorType){
3          val lectura=event.values[0]
4          Log.d("onSensorChanged", "Lectura: ${lectura}")
5      }
6  }
7
8  override fun onPause() {
9      super.onPause()
10     sensorManager.unregisterListener(this)
11 }
12
13 override fun onResume() {
14     super.onResume()
15     sensor?.also{pressure ->
16         sensorManager.registerListener(this, pressure, SensorManager.SENSOR_DELAY_NORMAL)
17     }
18 }
```


Proyecto siames

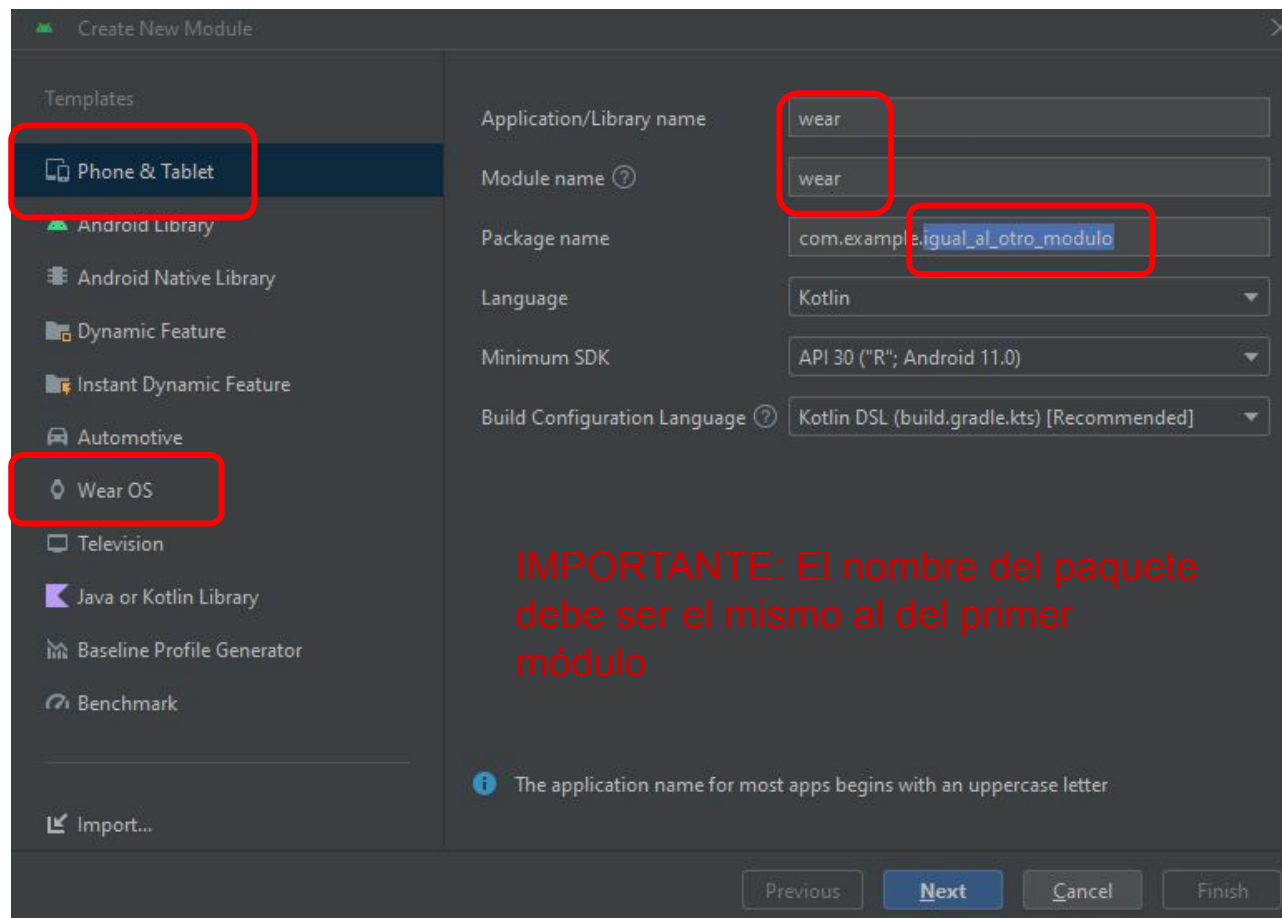
Le ponemos nombre al módulo acorde si es para phone o wear

A un proyecto ya creado, agregar un segundo módulo:

File > New > New Module

si el proyecto es para celular, agregamos un módulo wear

si es para wear, agregamos un módulo phone



Herencias de las clases

Nota: El Coroutine solo se necesita en el dispositivo que iniciará la conexión

```
1  class MainActivity : AppCompatActivity(),  
2      CoroutineScope by MainScope(),  
3      DataClient.OnDataChangedListener,  
4      MessageClient.OnMessageReceivedListener,  
5      CapabilityClient.OnCapabilityChangedListener  
6  {
```

Crear banderas

El botón solo va en el dispositivo que iniciará la conexión

```
1  lateinit var conectar: Button
2  var activityContext: Context?=null
3
4  private var deviceConnected: Boolean=false;
5  private val PAYLOAD_PATH="/APP_OPEN"
6  lateinit var nodeID:String
7
8  override fun onCreate(savedInstanceState: Bundle?) {
9      super.onCreate(savedInstanceState)
10     setContentView(R.layout.activity_main)
11     activityContext=this
12     conectar=findViewById(R.id.boton)
13
14     conectar.setOnClickListener{
15         if(!deviceConnected){
16             val tempAct:Activity=activityContext as MainActivity
17             getNodes(tempAct)
18         }
19     }
20 }
```

Obtener el id del nodo del dispositivo (sólo en el disp que inicia la conexión)

```
1 private fun getNodes(context: Context){
2     launch(Dispatchers.Default){
3         val nodeList=Wearable.getNodeClient(context).connectedNodes
4         try {
5             val nodes= Tasks.await(nodeList)
6             for (node in nodes){
7                 Log.d("NODO", node.toString())
8                 Log.d("NODO", "El id del nodo es: ${node.id}")
9                 nodeID=node.id
10                deviceConnected=true
11            }
12        }catch (exception: Exception){
13            Log.d("Error en el nodo", exception.toString())
14        }
15    }
16 }
```

Listeners (En ambos dispositivos)

```
1  override fun onPause(){
2      super.onPause()
3      try{
4          Wearable.getDataClient(activityContext!!).removeListener(this)
5          Wearable.getMessageClient(activityContext!!).removeListener(this)
6          Wearable.getCapabilityClient(activityContext!!).removeListener(this)
7      }catch(e: Exception){
8          e.printStackTrace()
9      }
10 }
11
12 override fun onResume() {
13     super.onResume()
14     try {
15         Wearable.getDataClient(activityContext!!).addListener(this)
16         Wearable.getMessageClient(activityContext!!).addListener(this)
17         Wearable.getCapabilityClient(activityContext!!)
18             .addListener(this, Uri.parse("wear://"), CapabilityClient.FILTER_REACHABLE)
19     }catch(e: Exception){
20         e.printStackTrace()
21     }
22 }
```

Enviar mensaje

```
1  private fun sendMessage(){
2      val sendMessage= Wearable.getMessageClient(activityContext!!)
3          .sendMessage(nodeID, PAYLOAD_PATH, "HOLA K ASE".toByteArray())
4          .addOnSuccessListener {
5              Log.d("sendMessage", "Mensaje enviado correctamente")
6          }
7          .addOnFailureListener { e->
8              Log.d("sendMessage", "Error al enviar mensaje ${e.message}")
9          }
10 }
```

Recibir mensajes

```
1  override fun onMessageReceived(ME: MessageEvent) {  
2      Log.d("onMessageReceived", ME.toString())  
3      Log.d("onMessageReceived", "ID del nodo ${ME.sourceNodeId}")  
4      Log.d("onMessageReceived", "Payload: ${ME.path}")  
5      val message=String(ME.data, StandardCharsets.UTF_8)  
6      Log.d("onMessageReceived", "Mensaje: ${message}")  
7  }
```


Peticiones http

```
1 fun get(url: String) {
2     // Crear un cliente de OkHttp
3     val client = OkHttpClient()
4
5     // Construir la petición
6     val request = Request.Builder()
7         .url(url)
8         .build()
9
10    // Ejecutar la petición en un hilo aparte
11    client.newCall(request).enqueue(object : okhttp3.Callback {
12        override fun onFailure(call: okhttp3.Call, e: IOException) {
13            // Manejo de error
14            Log.d("FETCH", "Error: ${e.message}")
15        }
16
17        override fun onResponse(call: okhttp3.Call, response: okhttp3.Response) {
18            response.use {
19                if (!response.isSuccessful) {
20                    Log.d("FETCH", "Error en la respuesta: ${response.code}")
21                } else {
22                    // Aquí se maneja la respuesta, por ejemplo, convertirla en String
23                    val responseData = response.body?.string()
24                    Log.d("FETCH", "Respuesta: $responseData")
25                }
26            }
27        }
28    })
29 }
```

```
1  val client = OkHttpClient()
2  val JSON = "application/json; charset=utf-8".toMediaTypeOrNull()
3  val body = jsonBody.toRequestBody(JSON)
4
5  val request = Request.Builder()
6      .url(url)
7      .post(body)
8      .build()
```